Flighty-Inspired Design System — SwiftUI Prompt Pack

A reverse-engineered prompt for building an intelligent iOS app with Flighty's visual and interaction language. Paste any section (or the whole thing) into Cursor / Claude Code / Xcode's AI to generate matching SwiftUI.

---

1. Design Philosophy (the "why")

Flighty doesn't look good because of colors and fonts. It looks good because every visual decision enforces one rule: **surface the single most useful piece of information for the user's current moment, and nothing else.** Everything else in the spec flows from this.

Four enforcing principles:

1. **Airport-board inheritance.** Density > decoration. One line = one thing. Monospaced numerics. Status is communicated by color and short uppercase labels (`ON TIME` , `DELAYED` , `BOARDING`), never buried in a sentence.
2. **Smart states over screens.** Flighty has ~15 context-aware states that rewrite the main card based on time, location, and data signals. Your app should do the same — the UI morphs as context changes, instead of making the user navigate.
3. **"Boringly obvious."** If the user has to think, you failed. Progressive disclosure: show 1–3 primary facts; tap for the rest.
4. **Offline-first, truthful live.** Pre-compute everything. A "LIVE" dot or countdown must only be shown when it's actually live — fake liveness erodes trust. When offline, degrade gracefully with cached + predicted values.

Your mantra when building a screen: *What does the user need to see right now, and what are they implicitly asking when they open this view?* Build that. Ship the rest behind a tap.

---

2. Color System

Airport signage color grammar. Colors have **meaning**, not flavor.

```
extension Color {
    // ── Status (semantic, never decorative) ──────────────────────
    static let statusGood      = Color(hex: "#10B981") // on time / success / hea
    static let statusWarning   = Color(hex: "#F59E0B") // delayed / attention / t
    static let statusBad       = Color(hex: "#EF4444") // cancelled / error / cri
    static let statusActive    = Color(hex: "#3B82F6") // boarding / in-progress
    static let statusNeutral   = Color(hex: "#6B7280") // scheduled / idle / info

    // ── Surface (dark-mode-first) ────────────────────────────────
    static let surfaceBase     = Color(hex: "#000000") // app background (true bl
    static let surfaceCard     = Color(hex: "#111114") // primary card
    static let surfaceCardAlt  = Color(hex: "#1C1C1E") // nested card
    static let surfaceDivider  = Color.white.opacity(0.06)

    // ── Content ──────────────────────────────────────────────────
    static let contentPrimary   = Color.white
    static let contentSecondary = Color.white.opacity(0.60)
    static let contentTertiary  = Color.white.opacity(0.35)

    // ── Accent ───────────────────────────────────────────────────
    static let accent           = Color(hex: "#5E8EFF") // app-specific, used spa
}
```

Rules:

- Dark mode is the canonical design. Light mode is a port, not the origin.
- Status colors only appear on status elements (badges, progress fills, numeric deltas). Never use them as decoration.
- Gradients: restricted to large background flourishes (passport pages, share cards). Never on buttons or cards.

---

3. Typography

Two typefaces do all the work. San Francisco Rounded for hierarchy and glanceable data. SF Mono for any number that might change or that the user will compare.

```
enum AppType {
    // Display: the one big fact on a screen
    static let display    = Font.system(size: 48, weight: .bold, design: .rounded
    // Metric: prominent numbers (countdowns, counts, times)
    static let metric     = Font.system(size: 32, weight: .bold, design: .monospa
    // Title: section & card headers
```

```
    static let title      = Font.system(size: 22, weight: .semibold, design: .rou
    // Identifier: codes, gates, IDs (stands out; comparable)
    static let identifier = Font.system(size: 18, weight: .bold, design: .monospa
    // Body
    static let body       = Font.system(size: 16, weight: .regular, design: .defa
    // Label / caption: secondary facts, uppercase status
    static let label      = Font.system(size: 13, weight: .semibold, design: .rou
    // Micro: tertiary, uppercase, letter-spaced
    static let micro      = Font.system(size: 11, weight: .semibold, design: .rou
}
```

Rules:

- Any number a user might compare or watch change → **monospaced**. Times, counts, percentages, IDs.
- Any number that is a one-off noun → rounded/default (e.g., "$42 total").
- Uppercase + letter-spacing (`.tracking(1.2)`) is reserved for status labels. Never use it for body copy.

---

4. Spacing, Radius, Elevation

```
enum Space {
    static let xs:  CGFloat = 4
    static let s:   CGFloat = 8
    static let m:   CGFloat = 12
    static let l:   CGFloat = 16   // default card padding
    static let xl:  CGFloat = 24
    static let xxl: CGFloat = 32
}

enum Radius {
    static let chip: CGFloat = 8
    static let card: CGFloat = 16   // default card
    static let hero: CGFloat = 24   // full-width hero blocks
    static let pill: CGFloat = 999
}
```

Rules:

- **No shadows** on dark surfaces. Separation comes from a subtle fill step (`#111114` → `#1C1C1E`) or a 1px inner divider at 6% white.

- Cards are edge-to-edge with 16pt horizontal page gutters.
- Vertical rhythm between cards: 12pt. Between sections: 24pt.

---

5. The Smart-State Engine (the core idea to steal)

This is Flighty's secret. It generalizes to any intelligent app.

Instead of one static home screen, define a **finite set of contextual states** and a single deterministic engine that picks the right state and renders it. The user never has to ask "what should I do now?" — the app already knows.

```
// 1) Define states as an enum — exhaustive, named by moment, not by feature
enum AppState {
    case idle                   // nothing relevant happening
    case upcoming(hours: Int)   // approaching something
    case imminent               // act now
    case active                 // in-progress, live
    case completing             // wrapping up
    case done                   // just finished, show outcome
    case recovering             // something went wrong, what to do next
}

// 2) A resolver that reads signals and returns the state
struct StateResolver {
    func resolve(_ signals: ContextSignals, now: Date = .now) -> AppState { ... }
}

// 3) One view, state-driven content
struct PrimaryCard: View {
    let state: AppState
    var body: some View {
        switch state {
        case .idle:              IdleContent()
        case .upcoming(let h):   UpcomingContent(hours: h)
        case .imminent:          ImminentContent()
        case .active:            LiveContent()
        case .completing:        CompletingContent()
        case .done:              OutcomeContent()
        case .recovering:        RecoveryContent()
        }
    }
}
```

Rules for state content:

- Each state shows **exactly one primary fact, one secondary fact, one action.** That's it.
- The primary fact uses `display` or `metric` type. The secondary uses `body` at `.secondary`. The action is a single text button.
- Transitions between states animate with `.spring(response: 0.4, dampingFraction: 0.85)` and a `contentTransition(.numericText())` on any numeric.

---

6. Component Library

6.1 StatusBadge

```
struct StatusBadge: View {
    enum Kind { case good, warning, bad, active, neutral }
    let kind: Kind
    let text: String   // always uppercase, ≤ 10 chars

    var body: some View {
        Text(text.uppercased())
            .font(AppType.micro)
            .tracking(1.2)
            .padding(.horizontal, 8)
            .padding(.vertical, 4)
            .background(kind.color.opacity(0.18))
            .foregroundStyle(kind.color)
            .clipShape(Capsule())
    }
}
```

6.2 Primary Card

The workhorse. Full-bleed card with a headline row, a metric row, and an optional progress/route visualization.

```
┌──────────────────────────────────────────────┐
│  [Identifier]   →   [Target]     [StatusBadge]│  ← headline row
│                                              │
│  [Big Metric, display type]                  │  ← primary fact
│  [Secondary fact, body.secondary]            │
│                                              │
│  [Progress / route viz — optional]           │
```

```
│                                                  │
│  [Action text button]                            │
└──────────────────────────────────────────────────┘
```

Padding `Space.l`. Background `surfaceCard`. Radius `Radius.card`.

6.3 Live Progress (linear)

Thin 3pt line, 0.3 opacity track, full-opacity accent fill, a filled circle marker at the head. Numeric labels sit **outside** the bar (left/right) in `identifier` type.

6.4 Live Progress (circular, compact)

For Dynamic Island / widgets. 6pt stroke, `squareCap` off (rounded). Progress text in the center, `metric` font.

6.5 Timeline Rows

A vertical dotted line on the left, each event is a row with: dot → time (`identifier`) → title (`body`) → optional right-aligned status chip. Past rows go to `contentTertiary`; current row stays `contentPrimary`; future rows `contentSecondary`.

6.6 Split Metric Row

Two or three metrics side by side, separated by a 1pt vertical divider. Label above (`micro`, uppercase, tracked) and value below (`metric`). This is the "stat strip" Flighty uses in Passport views.

6.7 Bottom Sheet

Default iOS sheet with `.presentationDetents([.medium, .large])`, grabber visible, corner radius 24pt, background `surfaceCard`. Sheets are the only place where progressive disclosure lives — primary screens never scroll to reveal hidden complexity.

---

7. Layout Patterns

7.1 Home = Map + List

Flighty's main screen is a muted custom map occupying the top ~40% with the active list overlapping it on a floating card. If your app has a spatial or temporal dimension, use this pattern: **context visualization on top, actionable list beneath.**

7.2 Detail = Vertical Scroll of Cards

A detail view is a stack of typed cards. Each card is one concern (status, timeline, connection, history, share). No nested tabs. No segmented controls hiding content.

7.3 Tab Bar = 3–5 nouns

Use standard iOS tab bar. Tabs are **nouns, not verbs** (Home, History, Passport, Settings). Never put a "+" in the tab bar; the "+" lives as a prominent button on the Home screen.

---

8. Motion & Haptics

- **Purposeful only.** If an animation doesn't communicate state change, remove it.
- **Numeric changes:** `contentTransition(.numericText())`.
- **State transitions:** `.spring(response: 0.4, dampingFraction: 0.85)`.
- **Long-running progress (in-flight-style):** `.linear` over the actual remaining duration, not a fake 1s ease.
- **Haptics:** `.sensoryFeedback(.success, trigger: ...)` on completion; `.warning` on a negative state change; `.selection` on tab taps. Never on scroll.

---

9. Live Activities & Widgets (first-class, not afterthought)

If your app has **any** time-bounded event (a workout, a delivery, a meeting, a cook timer, a trade, a drive), it gets a Live Activity and a Lock Screen widget. Treat them as the primary surface for active states — the user should almost never need to open the app once something is live.

Pattern (copy Flighty):

- **Compact leading:** the single most identifying token (a code, a glyph).
- **Compact trailing:** the most useful dynamic number (countdown, ETA, delta).
- **Expanded:** headline row + metric row + action(s).
- **Minimal (stacked):** one glyph.

Always support: Lock Screen, Home Screen widget (small + medium), StandBy variant, Watch complication if the domain warrants.

10. Data Presentation Rules (the “pack, wrap, color” doctrine)

Ryan Jones’s internal rule: data should be **packed** (no orphan numbers), **wrapped** (contextualized with a label or delta), and **colored** (pre-interpreted for the user).

Examples:

Raw	Packed / Wrapped / Colored
`23`	`23 min late` with orange `DELAYED` badge
`0.84`	`84% complete` with a progress arc
`14:32`	`Boards in 12 min` with active blue badge
`-2.4`	`▼ 2.4%` in red

If the user has to do math or interpretation, you haven’t finished designing the cell.

---

11. Iconography

- SF Symbols only. Weight `.medium` or `.semibold`. Never `.black`.
- Custom illustrations (planes, maps, passport art) are the one place Flighty permits painterly detail — gradients, multi-color, depth. Use the same rule: illustrations only on **celebratory / identity** surfaces (app icon, passport, share cards). Never in the main UI.

---

12. Ready-to-Paste Prompt Block

Copy the block below into Cursor, Claude Code, or Xcode AI when starting a new view:

> Build a SwiftUI view in the “Flighty design language.” Rules:
>
> - Dark-mode-first, true-black background (`#000`), cards `#111114` with 16pt radius, 16pt internal padding, 12pt between cards, 16pt page gutters. No shadows.
> - Typography: SF Rounded for hierarchy, SF Mono for any comparable/changing number. Status labels are UPPERCASE with 1.2pt tracking in `micro` size.
> - Color has semantic meaning only: `#10B981` good, `#F59E0B` warning, `#EF4444` bad,

`#3B82F6` active, `#6B7280` neutral. No decorative color.

- Each primary view shows at most one headline, one big metric, one secondary fact, one action. Push everything else into a `.sheet` with detents `[.medium, .large]`.
- If the view represents a time-bounded state, drive it through an `AppState` enum resolver (states: idle, upcoming, imminent, active, completing, done, recovering). Switch on the state to render content. Use `.contentTransition(.numericText())` on numerics and `.spring(response: 0.4, dampingFraction: 0.85)` on state changes.
- Status is always communicated with a `StatusBadge` (colored capsule, 18% background tint, full-color text). Never via a sentence.
- No segmented controls, no nested tabs, no decorative gradients, no shadows on dark surfaces.
- If the view involves a live event, also generate a matching `ActivityAttributes` + `ActivityConfiguration` scaffold for a Live Activity (compact leading/trailing, expanded, minimal) that mirrors the in-app primary card.

---

13. What to steal, per feature

Flighty feature	General pattern	Use it for
15 smart states	State-driven primary card	Any context-aware home screen
Live Activities	Primary surface for active events	Anything with a countdown or progress
Color-coded status	Semantic-color-only palette	All status communication
Airport board density	Monospaced metric rows	Any data-heavy list
Passport page	Shareable year-in-review artwork	Retention / identity / virality
Route progress	Linear + circular progress twin	Any linear journey
Connection assistant	“Tight / fine / comfortable” banner with recommended action	Any hand-off or transition

Offline-first	Pre-compute + cache + graceful degrade	Anything field-deployed

---

14. Things to explicitly avoid

- Blurred glass card-on-photo hero sections. (Not Flighty.)
- Colored gradient buttons. (Not Flighty.)
- Decorative emoji in UI. (Not Flighty.)
- Chat-bubble "AI assistant" panels inside the main UI. (Flighty's intelligence is invisible — the state engine is the AI. Keep yours invisible too.)
- Skeleton shimmer loaders. Show the last-known value dimmed instead; that's the offline-first way.
- Onboarding carousels longer than 2 screens.

Spec compiled from Flighty's Apple Design Award 2023 writeup, Ryan Jones interviews, Verena Ortlieb's design notes, and Blake Crosley's implementation guide. The intelligence pattern (smart-state engine + invisible AI + live-first surfaces) is the part worth stealing for any "intelligent app."